

Deploy DOJOBID — Path A (Vercel + Railway + Neon)

Purpose: Step-by-step production deployment (managed hosting) **Audience:** Developers and operators deploying DOJOBID **Stack:** Vercel (web) + Railway (API) + Neon (database) + GoDaddy DNS **Document date:** June 7, 2026

This guide deploys **DOJOBID** using managed hosting (no VPS): ComponentPlatformURL example
__TABLE_SEP__ **Web** (apps/web) [Vercel](#) https://yourdomain.com **API**
(apps/api) [Railway](#) https://api.yourdomain.com **Database** [Neon](#) ep-proud-art-aqlgk6ni · us-east-1 ·
database neondb **Domain / DNS** GoDaddy (or any registrar) Points web → Vercel, API → Railway **Render**
alternative: Railway and [Render](#) are both fine for the API. See [Appendix — Deploy API on Render instead of Railway](#). **Mobile apps** (Expo) are built separately. They only need the public API URL
(EXPO_PUBLIC_API_URL).

Screenshots in this guide are annotated reference UI mockups stored in docs/deploy-screenshots/path-a/. They show the exact fields, values, and buttons for DOJOBID. Regenerate them with node render-deploy-screenshots.mjs from the docs/ folder. Your provider dashboards may look slightly different after UI updates.

FigureFileStep __TABLE_SEP__ 1 neon-01-dashboard.png Neon project overview 2 neon-02-connection-pooled.png Neon pooled connection string 3 railway-01-github-deploy.png Railway deploy from GitHub 4 railway-02-build-settings.png Railway build/start commands 5 railway-03-variables.png Railway environment variables 6 railway-04-networking-domain.png Railway custom domain 7 railway-05-health-check.png Railway health endpoint 8–11 vercel-01 ... vercel-04 Vercel import, monorepo, env, domains 12 godaddy-01-dns.png GoDaddy DNS records 13–14 verify-01, verify-02 Production smoke tests

Architecture

```
``text Internet | | https://yourdomain.com → Vercel (Next.js) | | | browser calls
| | | | | | |
| | | | | https://api.yourdomain.com → Railway (NestJS) | |
| | | | |
| | | | | Neon · ep-proud-art-aqlgk6ni (us-east-1) · neondb `
```

Prerequisites on your PC

1. Node.js ≥ 20
1. Git repo pushed to **GitHub** (Vercel and Railway connect via GitHub)

1. GoDaddy domain (or any DNS provider)

1. Neon account

Estimated monthly cost (starter)

1. Neon: free tier often enough for early launch

1. Vercel: free hobby tier for personal/small projects

1. Railway: ~\$5+ usage-based (check current pricing)

1. GoDaddy: domain renewal only (no VPS required)

Phase 1 – Neon database

You already have a Neon project configured for local development in `apps/api/.env`. Production (Railway) uses the **same Neon project** – no second database required for your first launch.

Your Neon instance (current)

SettingValue `__TABLE_SEP__` Endpoint ID `ep-proud-art-aqlgk6ni` Region `us-east-1` (AWS)
Database `neondb` Role `neondb_owner` Direct host (local dev) `ep-proud-art-aqlgk6ni.c-8.us-east-1.aws.neon.tech` Pooled host (Railway – recommended) `ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech` Local config `apps/api/.env` → `DATABASE_URL`

Security: The full connection string (including password) lives in `apps/api/.env` on your PC only. Do not commit that file or paste the password into Git, Vercel, or public docs.

Step 1. Confirm access in the Neon console

1. Sign in at <https://console.neon.tech>.

1. Open the project that contains endpoint **ep-proud-art-aqlgk6ni** (region **US East 1**).

1. Confirm the database **neondb** exists and migrations have been applied from local dev.

If you later want isolated prod data, create a **branch** in this same Neon project rather than a whole new account. ![Figure 1 – Neon console: confirm project endpoint ep-proud-art-aqlgk6ni, database neondb, region US East 1](#) What to look for (Figure 1): Project overview shows endpoint **ep-proud-art-aqlgk6ni**, database **neondb**, and region **US East 1**. If you see a different

endpoint, open the project that matches your `apps/api/.env` file.

Step 2. Copy the connection string for Railway

In Neon → **Connection details** for `ep-proud-art-aqlgk6ni` :

1. Select **Pooled connection** (recommended for Railway – avoids connection limit errors).

1. Copy the URI. It should match this shape (password hidden):

```
`text postgresql://neondb_owner:YOUR_PASSWORD@ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require`
```

1. Keep `sslmode=require` .

Alternatively, copy the value already in `apps/api/.env` and change the host from:

1. `ep-proud-art-aqlgk6ni.c-8.us-east-1.aws.neon.tech` (direct – fine for local Prisma)

to:

1. `ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech` (pooled – use on Railway)

Save the pooled string in a password manager for Railway **Variables**. Do not commit it to Git.

! [Figure 2 – Neon Connection details: select Pooled connection and copy the URI](#) What to look for (Figure 2): **Connection details** → **Pooled connection** selected (not Direct). Host must include **-pooler** before `.c-8.us-east-1.aws.neon.tech` . Click **Copy connection string** and paste into Railway (Step 6).

Step 3. Apply the database schema (from your PC)

Your schema may already be applied from local development. Safe to run again before go-live:

```
`powershell cd C:\Users\robbyu\cursor\projects\ContractorBidder`
```

Uses DATABASE_URL from apps/api/.env (your Neon instance)

`npm run db:generate npm run prisma:deploy --workspace apps/api` ` Or set the pooled URL explicitly for one shell session: ``powershell cd`

`C:\Users\robbyu\cursor\projects\ContractorBidder``

```
$env:DATABASE_URL="postgresql://neondb_owner:YOUR_PASSWORD@ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require" npm run db:generate npm run prisma:deploy --workspace apps/api`
```

Replace `YOUR_PASSWORD` with the password from Neon or `apps/api/.env` .

Optional – **staging only** (do not seed real production): ``powershell npm run db:seed --workspace`

```
apps/api ` Demo logins after seed: homeowner@example.com / contractor@example.com /
password Password123!
```

Phase 2 – Deploy the API on Railway

Railway runs the NestJS API from the **monorepo root** so workspace packages (`@contractor-bidder/types`) resolve correctly.

Step 4. Create a Railway project

1. Sign in at <https://railway.app>.
1. New Project → Deploy from GitHub repo.
1. Authorize GitHub and select your **ContractorBidder** repository.
1. Railway creates a service from the repo.

! [Figure 3 – Railway New Project: Deploy from GitHub repo, select ContractorBidder](#) What to look for (Figure 3): Choose **Deploy from GitHub repo** (not Empty Project). Select your **ContractorBidder** repository and confirm Railway has read access to the repo.

Step 5. Configure the API service

Open the service → **Settings**: SettingValue `__TABLE_SEP__` **Root Directory** / (repo root – leave blank/default) **Watch Paths** (optional) `apps/api/`, **packages/** **Build command**: ``bash npm ci && npm run db:generate --workspace apps/api && npm run build --workspace apps/api`` **Start command**: ``bash npm run prisma:deploy --workspace apps/api && npm run start:prod --workspace apps/api`` Running `prisma migrate deploy` on each deploy keeps Neon in sync when you ship schema changes. ! [Figure 4 – Railway Settings: build and start commands for the monorepo API](#) What to look for (Figure 4): **Root Directory** stays at repo root (blank or /). Paste the **Build command** and **Start command** exactly as shown in the table above. Save settings before deploying.

Step 6. Set Railway environment variables

In the service → **Variables**, add: ! [Figure 5 – Railway Variables: DATABASE_URL, API_PORT, JWT secrets, CORS](#) What to look for (Figure 5): Open **Variables** and add every key from the block below. **DATABASE_URL** uses the pooled Neon host. **API_PORT** must be `${{PORT}}` (Railway variable reference, not the number 4000). ``env``

Pooled URL for ep-proud-art-aqlgk6ni (same Neon project as apps/api/.env)

```
DATABASE_URL=postgresql://neondb_owner:YOUR_PASSWORD@ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require
```

Railway injects PORT – bind the NestJS

app to it

API_PORT=\${PORT}

Set after Vercel is live (Step 11); use Railway URL temporarily for first deploy

API_CORS_ORIGIN=https://yourdomain.com,https://www.yourdomain.com

JWT_ACCESS_SECRET=PASTE_LONG_RANDOM_STRING_1 JWT_REFRESH_SECRET=PASTE_LONG_RANDOM_STRING_2

JWT_ACCESS_TTL=900s JWT_REFRESH_TTL=30d PAYMENTS_ENABLED=false MESSAGING_GROUP_VISIBLE=false

PROFILE_REQUIRE_VERIFICATION=false JOBS_MAX_PHOTOS=4 AI_JOB_DESCRIPTION_ENABLED=true

GEMINI_API_KEY= GEMINI_MODEL=gemini-2.0-flash STRIPE_SECRET_KEY= STRIPE_WEBHOOK_SECRET=

AWS_REGION=us-east-1 MEDIA_S3_BUCKET= MEDIA_S3_ENDPOINT= MEDIA_PUBLIC_BASE_URL= `Generate JWT secrets (PowerShell): `powershell [Convert]::ToBase64String((1..48 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]]) `Run twice for the two JWT values. **Important – job photos on Railway** Railway's filesystem is **ephemeral**. Without S3, uploads go to `.dev-media` on the container and **disappear on redeploy**. For production, configure object storage before launch:

1. **AWS S3**, or

1. **Cloudflare R2** (S3-compatible – set `MEDIA_S3_ENDPOINT`)

Leave `MEDIA_S3_BUCKET` empty only for short-lived demos. After S3/R2 is configured, set: ``env MEDIA_PUBLIC_BASE_URL=https://api.yourdomain.com/api/v1/dev-media `` (or your CDN URL if using public bucket/CloudFront) Redeploy after changing variables.

Step 7. Add a custom API domain on Railway

1. **Settings** → **Networking** → **Generate Domain** first if you have not already (gives a `*.up.railway.app` URL for testing).

1. **Settings** → **Networking** → **Custom Domain** → add `api.yourdomain.com` .

1. Railway shows a **CNAME target** (e.g. `something.up.railway.app`).

1. Leave this tab open – you will create the DNS record in Phase 4.

!Figure 6 – Railway Networking: public domain and custom domain `api.yourdomain.com` What to look for (Figure 6): Copy the **CNAME target** Railway displays. You will paste it into GoDaddy as a `api` CNAME record (Step 14). !Figure 7 – Railway health check: `GET /api/v1/health` returns `status ok` What to look for (Figure 7): Before adding DNS, open `https://YOUR-SERVICE.up.railway.app/api/v1/health` in a browser. You should see JSON with `"status":"ok"`. If not, check Railway deploy logs and `DATABASE_URL`. Test after DNS propagates: `text https://api.yourdomain.com/api/v1/health`

Phase 3 – Deploy the web app on Vercel

Step 8. Import the repo on Vercel

1. Sign in at <https://vercel.com>.
1. **Add New** → **Project** → import the same GitHub repository.
1. Configure the project:

SettingValue `__TABLE_SEP__` Framework `PresetNext.js` Root Directory `apps/web` Include source files outside of the Root DirectoryEnabled (required for monorepo workspace packages) !Figure 8 – Vercel import: connect GitHub repository ContractorBidder What to look for (Figure 8): **Add New** → **Project** → import the same GitHub repo used on Railway. Grant Vercel access if prompted. !Figure 9 – Vercel configure: Root Directory `apps/web`, include monorepo packages What to look for (Figure 9): Set **Root Directory** to `apps/web`. Enable **Include source files outside of the Root Directory** – required for `@contractor-bidder/types` and `@contractor-bidder/ui`.

Step 9. Set Vercel environment variables

In **Project** → **Settings** → **Environment Variables**: NameValueEnvironments `__TABLE_SEP__` `NEXT_PUBLIC_API_URL` `https://api.yourdomain.com/api/v1` Production (and Preview if you want staging) !Figure 10 – Vercel Environment Variables: `NEXT_PUBLIC_API_URL` for production What to look for (Figure 10): Add **`NEXT_PUBLIC_API_URL`** before the first production deploy. Use your Railway `*.up.railway.app` URL temporarily if `api.yourdomain.com` is not ready yet. **Changing `NEXT_PUBLIC_*` requires a new deploy** – Vercel bakes it in at build time.

Step 10. Deploy

1. Click **Deploy**.

1. Vercel runs `npm install` from the monorepo root and builds `apps/web`.

1. Open the *.vercel.app URL – you should see the login page.

1. Open browser DevTools → Network. Confirm API calls go to https://api.yourdomain.com/api/v1/... , not localhost:4000 .

If the web app still calls localhost, fix NEXT_PUBLIC_API_URL and Redeploy. See Figure 14 in Step 16.

Step 11. Add custom domains on Vercel

1. Project → Settings → Domains.

1. Add yourdomain.com and www.yourdomain.com .

1. Vercel shows required DNS records.

!Figure 11 – Vercel Domains: yourdomain.com and www with DNS instructions What to look for (Figure 11): Both domains should show Valid Configuration after DNS propagates. Copy Vercel's records into GoDaddy (Step 13).

Step 12. Update API CORS on Railway

Once you know the final web URLs, update Railway: `` env
API_CORS_ORIGIN=https://yourdomain.com,https://www.yourdomain.com,https://YOUR-PROJECT.vercel.app
`` Include the *.vercel.app URL if you use preview deployments. Redeploy the API service.

Phase 4 – GoDaddy DNS

In GoDaddy → My Products → your domain → DNS (or Manage DNS).

Step 13. Web → Vercel

Use the exact records Vercel shows in the Domains panel. Typical setup:

Type	Name	Value	Notes
A	@	76.76.21.21	Vercel apex (confirm in Vercel UI – IP can change)
CNAME	www	cname.vercel-dns.com	Or the hostname Vercel assigns Some registrars use ALIAS/ANAME for apex instead of A – follow Vercel's instructions for GoDaddy.

 !Figure 12 – GoDaddy DNS: A record for web, CNAME for www and api What to look for (Figure 12): Three records – @ A → Vercel IP, **www** CNAME → Vercel, **api** CNAME → Railway target from Figure 6. Do not point api

at Vercel.

Step 14. API → Railway

TypeNameValue __TABLE_SEP__ CNAME api Railway CNAME target from Step 7 Example: api → your-service.up.railway.app **Do not point api at Vercel or the apex A record.**

Step 15. Wait for propagation

DNS can take a few minutes to a few hours. Check: `\ powershell nslookup yourdomain.com nslookup api.yourdomain.com` Both should resolve. HTTPS is handled automatically by Vercel and Railway once DNS is correct.

Phase 5 – Production verification

Step 16. Smoke test

CheckURL / action __TABLE_SEP__ API health `https://api.yourdomain.com/api/v1/health` → `{"status":"ok",...}` API flags `https://api.yourdomain.com/api/v1/flags` → JSON feature flags Web loads `https://yourdomain.com` → login page Register / loginCreate account or use seeded demo (staging only) Post a jobGeocoding, description, photo upload CORSNo browser console errors on API calls from web RealtimeOpen job detail in two tabs – bids/messages update (Socket.IO) ![Figure 13 – Production API health on custom domain api.yourdomain.com](#) What to look for (Figure 13): Browser shows JSON `{"status":"ok",...}` at `https://api.yourdomain.com/api/v1/health` – confirms Railway + DNS + SSL. ![Figure 14 – Browser DevTools Network: API calls go to api.yourdomain.com, not localhost](#) What to look for (Figure 14): On `https://yourdomain.com`, open DevTools → Network. Login and browse jobs – every API request should target **api.yourdomain.com**, never `localhost:4000`.

Step 17. Mobile app (optional)

On your dev machine, `apps/mobile/.env`: `\ env`
`EXPO_PUBLIC_API_URL=https://api.yourdomain.com/api/v1` Restart Expo. Physical devices must reach the public HTTPS API (not a LAN IP). For App Store builds, set the same variable in EAS secrets / build profile.

Phase 6 – Deploying updates

Web (Vercel)

Push to the connected Git branch → Vercel auto-builds. If you change `NEXT_PUBLIC_API_URL`, trigger a **Redeploy** from the Vercel dashboard.

API (Railway)

Push to GitHub → Railway auto-builds and runs migrations (via start command). Manual migration from your PC (if needed): `\ powershell`


```
$env:DATABASE_URL="postgresql://neondb_owner:YOUR_PASSWORD@ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech/neondb?sslmode=require" npm run prisma:deploy --workspace apps/api`
```

Database (Neon)

Schema changes ship with API deploys (`prisma migrate deploy`). Backups and branching are managed in the Neon dashboard.

Troubleshooting

SymptomLikely causeFix
__TABLE_SEP__ API build fails on RailwayMonorepo / PrismaUse repo **root** as Root Directory; run `db:generate` before build
API won't startWrong DATABASE_URL Use pooled host `ep-proud-art-aqlgk6ni-pooler.c-8.us-east-1.aws.neon.tech`, `sslmode=require`, check Railway logs
API listens on wrong portAPI_PORT not setSet `API_PORT=${{PORT}}` on Railway
Unauthorized / CORS in browserMissing web originAdd exact `https://` URLs to `API_CORS_ORIGIN`; redeploy
API Web calls `localhost:4000` Stale NEXT_PUBLIC_API_URL Fix env on Vercel → **Redeploy**
Photos upload then vanishEphemeral Railway diskConfigure S3 or R2 (`MEDIA_S3_BUCKET`, credentials via Railway)
SSL / certificate errorsDNS not propagatedWait; confirm CNAME for `api`, Vercel records for `@ / WWW`
Neon connection limitsToo many connectionsUse **pooled** host (`...-pooler.c-8.us-east-1.aws.neon.tech`), not the direct endpoint
Health OK but login failsMigrations not appliedRun `prisma:deploy` locally or fix Railway start command
Useful URLs

1. Railway: service → **Deployments** → **View logs**
 1. Vercel: project → **Deployments** → **Build logs**
 1. Neon: **Monitoring** for connections and query load
-

Quick checklist

1. [] Neon **ep-proud-art-aqlgk6ni** – pooled DATABASE_URL saved for Railway (from dashboard or `apps/api/.env` + `-pooler` host)

1. [] `prisma migrate deploy` run against Neon

1. [] Railway API deployed; `API_PORT=${{PORT}}`; JWT secrets set

1. [] Railway health check: `/api/v1/health`

1. [] Vercel web deployed; `NEXT_PUBLIC_API_URL` points at production API

1. [] GoDaddy DNS: `@ / www` → Vercel, `api` → Railway CNAME

1. [] `API_CORS_ORIGIN` includes production web URL(s)

1. [] S3/R2 configured for job photos (recommended before real users)

1. [] Login, post job, and photo flow tested end-to-end

1. [] Demo seed passwords removed or not used in production

Appendix – Deploy API on Render instead of Railway

Render steps mirror Railway with different labels:

1. render.com → New → Web Service → connect GitHub repo.

1. **Root Directory:** leave blank (repo root).

1. **Runtime:** Node.

1. **Build Command:**

```
`bash npm ci && npm run db:generate --workspace apps/api && npm run build --workspace apps/api`
```

1. **Start Command** (Render injects `PORT` – map it to `API_PORT`):

```
`bash export API_PORT=$PORT && npm run prisma:deploy --workspace apps/api && npm run start:prod --workspace apps/api`
```

1. **Environment variables:** same list as Step 6 (omit `API_PORT` in the dashboard).

1. **Custom domain:** add `api.yourdomain.com` in Render → point GoDaddy **CNAME** `api` to Render's hostname.

Everything else in this guide (Neon, Vercel, DNS, CORS, media storage) stays the same.

Optional – **railway.toml** at repo root

If you prefer config in Git, create `railway.toml` : ``toml [build] builder = "NIXPACKS" buildCommand = "npm ci && npm run db:generate --workspace apps/api && npm run build --workspace apps/api" [deploy] startCommand = "npm run prisma:deploy --workspace apps/api && npm run start:prod --workspace apps/api" restartPolicyType = "ON_FAILURE" restartPolicyMaxRetries = 10``
Commit and push; Railway picks it up on the next deploy.

Related docs

1. PDF: `docs/Deploy-Vercel-Railway-Neon.pdf` (regenerate with `node generate-pdf.mjs deploy-vercel-railway-neon` from `docs/`)

1. Screenshots: `docs/deploy-screenshots/path-a/` (regenerate with `node render-deploy-screenshots.mjs` from `docs/`)

1. VPS + Neon (all-in-one server): `docs/Deploy-GoDaddy-Windows-Neon.md`

1. Environment variables: `.env.example` , `apps/api/.env.example`

1. Operations: docs/DOJOBID-Operators-Manual.md

1. Local development: root README.md`